Day 3 - Training Deep Networks & Evaluation

How to train a network well - and how to show honestly that it works

Annika Stiehl · Deep Learning · M.Sc. · SRH

Today

Time	
09:00–10:30	The training loop and its knobs: loss • class imbalance • optimisers (momentum, Adam) • learning rate & schedules
10:45–12:15	Generalisation: underfitting vs overfitting • the toolbox (weight decay, dropout, BatchNorm, early stopping, augmentation) • tuning & tracking your runs
13:00–14:30	Evaluation: splits & data leakage • confusion matrix & metrics (exercise) • seeds & confidence intervals
14:45–16:00	Lab 3: build a model and a training pipeline yourself • grid search • the right metric in the right place (CHB-MIT) • Q&A

💻 Today we code during the lecture: open `day3a_training_knobs`, `day3b_generalisation` and `day3c_evaluation` from the course repository in Colab (CPU is enough).

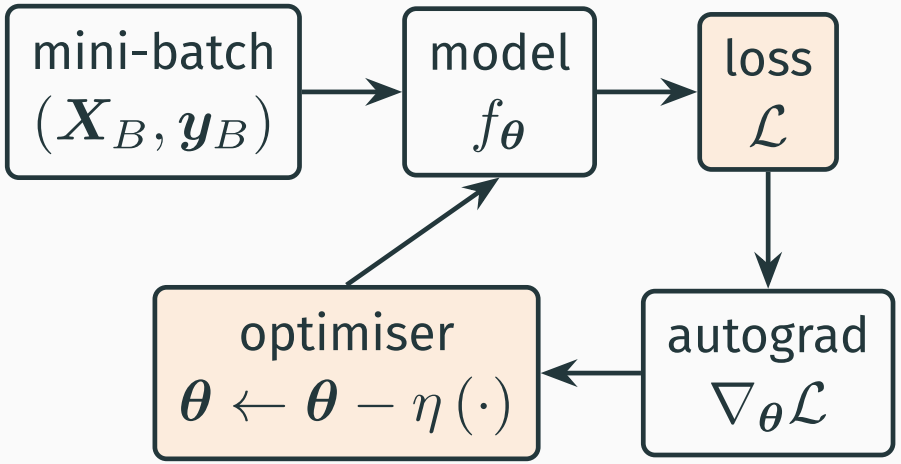
Last time

- MLP = affine layers + non-linearities; without non-linearity all layers collapse into one linear map
- Backpropagation = chain rule on the computation graph; one backward pass yields all gradients
- Autograd records the graph in the forward pass, `loss.backward()` fills every `.grad`

The training loop and its knobs

What sits where - and what you can change

Training a network = a handful of design decisions



regularisation, normalisation and initialisation shape *which* θ we find

- **Block 1**: which knobs does the training loop have - and what happens if I turn them? (loss, optimiser, learning rate)
- **Block 2**: how do we make it generalise? (regularisation, BatchNorm, init, tuning)
- **Block 3**: how do we know that it works? (splits, metrics, statistics)

The training loop: seven knobs

```
model   = nn.Sequential(nn.Linear(8, 64), nn.ReLU(),
                        nn.Linear(64, 2))          # ① model
loss_fn = nn.CrossEntropyLoss(weight=w)            # ② loss
opt     = torch.optim.AdamW(model.parameters(),    # ③ optimiser
                            lr=1e-3)               # ④ learning rate
sched   = torch.optim.lr_scheduler.CosineAnnealingLR(
              opt, T_max=20)                       # ⑤ LR schedule
loader  = DataLoader(train_ds, batch_size=64,
                     shuffle=True)                 # ⑥ batch size

for epoch in range(20):                            # ⑦ epochs
    for x, y in loader:
        loss = loss_fn(model(x), y)    # forward: how wrong are we?
        opt.zero_grad()
        loss.backward()                # backward: gradients (Day 2)
        opt.step()                     # update: change the weights
    sched.step()
```

- the loop from Day 2 - every knob is one line. This morning: ②, ③, ④, ⑤ - ⑥ ⑦ on the cheat sheet, ① from Day 4 on

🔑 Key takeaway

Training a network = choosing the knobs. You almost never change the loop itself.

② Which loss for which task?

task	example	output layer	loss in PyTorch
predict a **number** (regression)	house price, age	1 neuron, no activation	`nn.MSELoss()`
pick **one of K classes**	FashionMNIST (10 classes)	K neurons (logits)	`nn.CrossEntropyLoss()`
yes / no (binary)	seizure or not	1 neuron (logit)	`nn.BCEWithLogitsLoss()`

- the loss is one number: how wrong is the model on this batch? Small = good.
- cross-entropy from Day 2: a confident wrong answer costs a lot
- binary also works with 2 neurons + `CrossEntropyLoss` (that is what we do in the labs)

🔑 Key takeaway

The task decides the loss. You rarely invent your own.

② Pitfall: give the loss the raw scores (logits)

```
logits = model(x)                    # raw scores - NO softmax layer at the end of the model!
loss = F.cross_entropy(logits, y)    # softmax happens INSIDE the loss
probs = logits.softmax(dim=1)        # only if you want to show probabilities
```

- `CrossEntropyLoss` = softmax + cross-entropy in one step (numerically safer)

⚠️ What goes wrong if the model ends with nn.Softmax()?

The softmax is applied **twice**: logits (2, 1, 0) → softmax (0.67, 0.24, 0.09) → softmax again (0.45, 0.30, 0.25) - almost equal.

- even a perfect model can never reach loss 0 (3 classes: never below 0.55)
- small gradients → training is very slow
- no error message - that is why this bug is dangerous

② Class imbalance: weight the loss

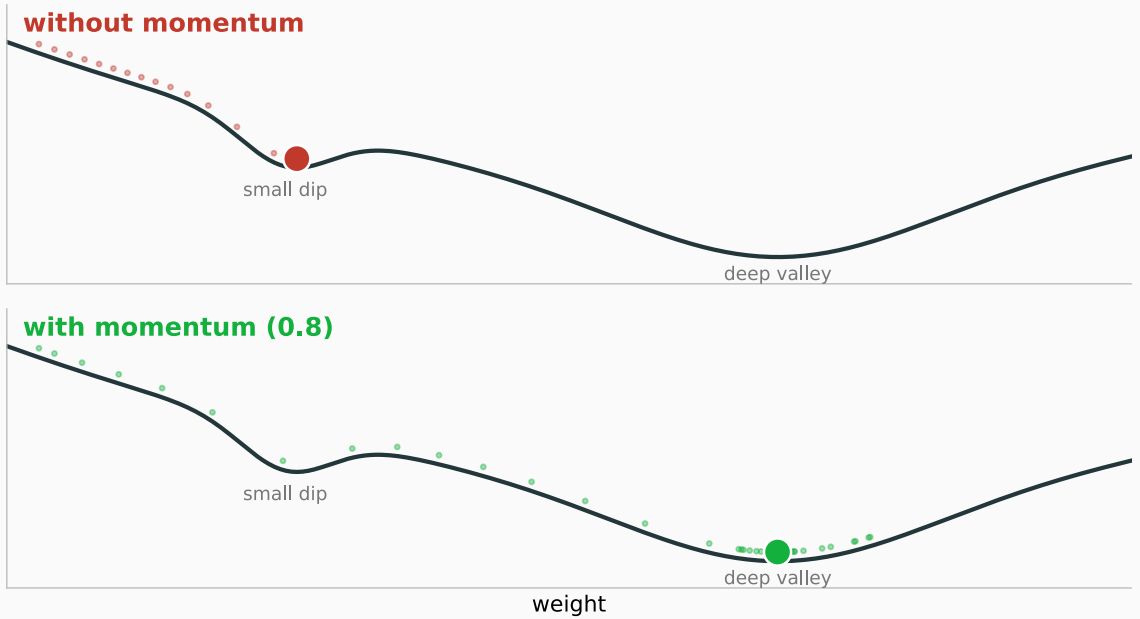
- chb01: **86 of 5081** windows are seizure (1.7 %)
- a model that always says "background" is right 98.3 % of the time - and finds no seizure at all
- the loss adds up all windows: 4995 background windows outvote 86 seizure windows
- fix: give every seizure window a larger **weight** - 4995 / 86 ≈ 58: one seizure window counts like 58 background windows

```
w = torch.tensor([1.0, 58.0])            # [background, seizure]
loss_fn = nn.CrossEntropyLoss(weight=w)
```

🧠 EEG corner

Seizure detection is extremely imbalanced: hours of background per minute of seizure. Weighting makes the model find more seizures - and raise more false alarms.
Its outputs are then no real probabilities any more, so 0.5 is not a special threshold (block 3).

③ Optimiser - Intuition: a ball rolling downhill



- **without momentum**: every step only looks at the slope right here → the ball stops in the first small dip
- **with momentum**: the ball keeps part of its speed → it rolls through the small dip into the deep valley (and swings back a little)

③ Optimiser - momentum = a ball with memory



- plain gradient descent only looks at the slope right here → in a narrow valley it zig-zags from wall to wall and crawls along the valley
- **momentum**: keep part of the last step - like a heavy ball rolling downhill
- the zig-zag cancels out, the steady direction adds up → faster and smoother
- in PyTorch it is one argument:

```
opt = torch.optim.SGD(model.parameters(),
                      lr=0.01, momentum=0.9)
```

`momentum=0.9` = "keep 90 % of the last step"

Try it: Why Momentum Really Works

distill.pub/2017/momentum - only the **first figure** at the top. Two sliders: step-size (= learning rate) and momentum.

1. set **momentum to 0** = plain gradient descent. How does the path look? How fast does it get to the minimum?
2. set **momentum to about 0.9**. What changes?
3. set **momentum to 0.99**. What happens now?

③ Adam: every weight gets its own step size

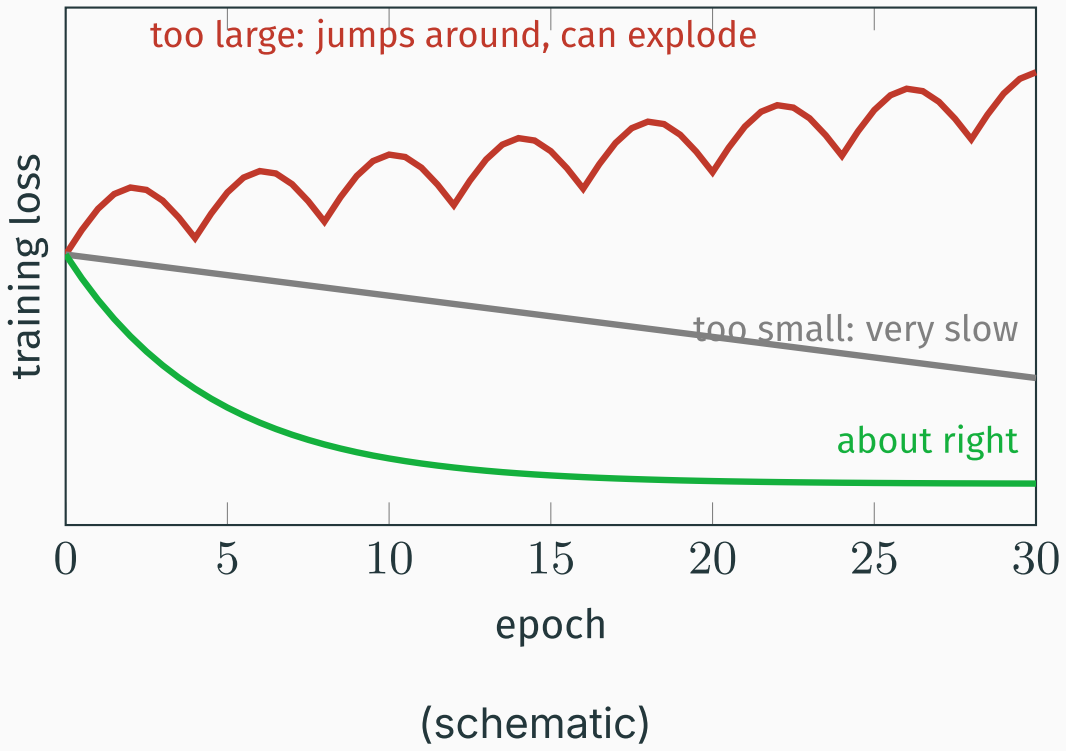
- SGD: one learning rate for all weights
- **Adam** watches every weight separately:
  - big, wild gradients → smaller steps
  - tiny gradients → bigger steps
  - plus momentum built in
- → works well with the default `lr=1e-3` - a good first choice
- **AdamW** = Adam + weight decay done right (weight decay: block 2). With weight decay → AdamW.

```
# swapping the optimiser = changing one line
params = list(model.parameters())
opt = torch.optim.SGD(params, lr=0.01, momentum=0.9)
opt = torch.optim.Adam(params, lr=1e-3)
opt = torch.optim.AdamW(params, lr=1e-3,
                        weight_decay=0.01)
```

🔑 Key takeaway

Start with AdamW and `lr=1e-3`. Try SGD + momentum if you have time to tune (often best for CNNs on images).

④ The learning rate: the most important knob



(schematic)

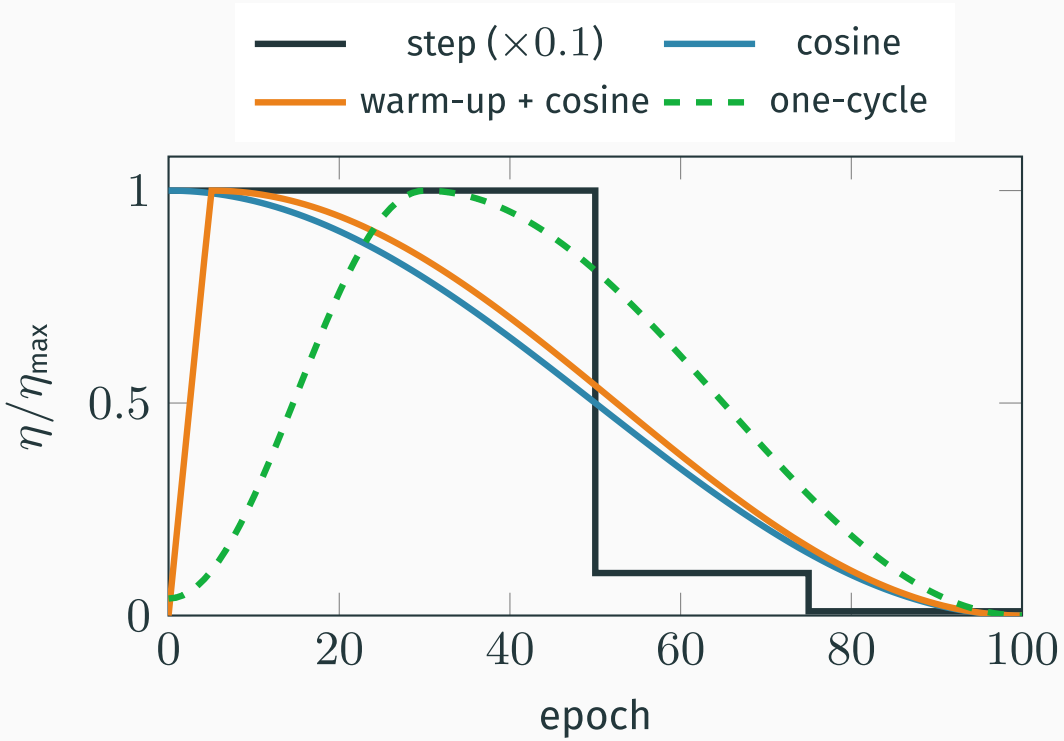
- **too large**: the steps jump over the valley - the loss jumps up and down, can become `nan`
- **too small**: every step is tiny - the loss falls, but it takes forever
- **about right**: falls fast, then flattens
- if you change only one thing, change the learning rate - in factors of about 3: 0.1, 0.03, 0.01, 0.003, ...

Try it: learning rate in TensorFlow Playground

playground.tensorflow.org - dataset Circle, activation **ReLU**, 2 hidden layers with 4 neurons

1. learning rate **0.03** (default). Press play. Test loss after about 200 epochs?
2. reset (↻), learning rate **3**. What happens?
3. reset, learning rate **0.0001**. What happens?
4. find the largest learning rate that still works well

⑤ Learning-rate schedules: big steps first, small steps later



- start: big steps → get close to the valley fast
- end: small steps → settle at the bottom
- **cosine**: smooth, a good default
- **warm-up**: start gently with a small learning rate (large models, transformers - Day 5)

```
opt = torch.optim.AdamW(model.parameters(), lr=1e-3)
sched = torch.optim.lr_scheduler.CosineAnnealingLR(
    opt, T_max=epochs)

for epoch in range(epochs):
    for x, y in loader:
        loss = loss_fn(model(x), y)
        opt.zero_grad()
        loss.backward()
        opt.step()
    sched.step()          # once per epoch
```

👥 Your turn: which knob?

#	symptom	which knob? what do you change?
1	the loss becomes `nan` after a few batches	
2	the loss falls very slowly - after 50 epochs it is still falling	
3	the loss jumps up and down a lot	
4	98 % accuracy on CHB-MIT - but not a single seizure found	
5	the model ends with `nn.Softmax()`, the loss is `CrossEntropyLoss`, training is very slow	
6	you predict the patient's age, use `CrossEntropyLoss` - and get an error	

👥 In pairs (5 min)

The knobs: ② loss · ③ optimiser · ④ learning rate · ⑤ LR schedule · ⑥ batch size

Cheat sheet: the knobs at a glance

	knob	where	good first choice	turn it when …
②	loss	`nn.CrossEntropyLoss()`	cross-entropy (classes), MSE (numbers)	imbalanced classes → `weight=`
③	optimiser	`torch.optim.AdamW(...)`	AdamW	CNNs on images: try SGD + `momentum=0.9`
④	learning rate	`lr=`	`1e-3` (Adam), `0.01`-`0.1` (SGD)	loss `nan` / jumping → smaller; very slow → larger
⑤	schedule	`lr_scheduler.CosineAnnealingLR`	cosine	the loss stops improving early
⑥	batch size	`DataLoader(batch_size=)`	32-128	GPU out of memory → smaller
⑦	epochs	`range(epochs)`	until the validation loss stops improving	→ early stopping (block 2)

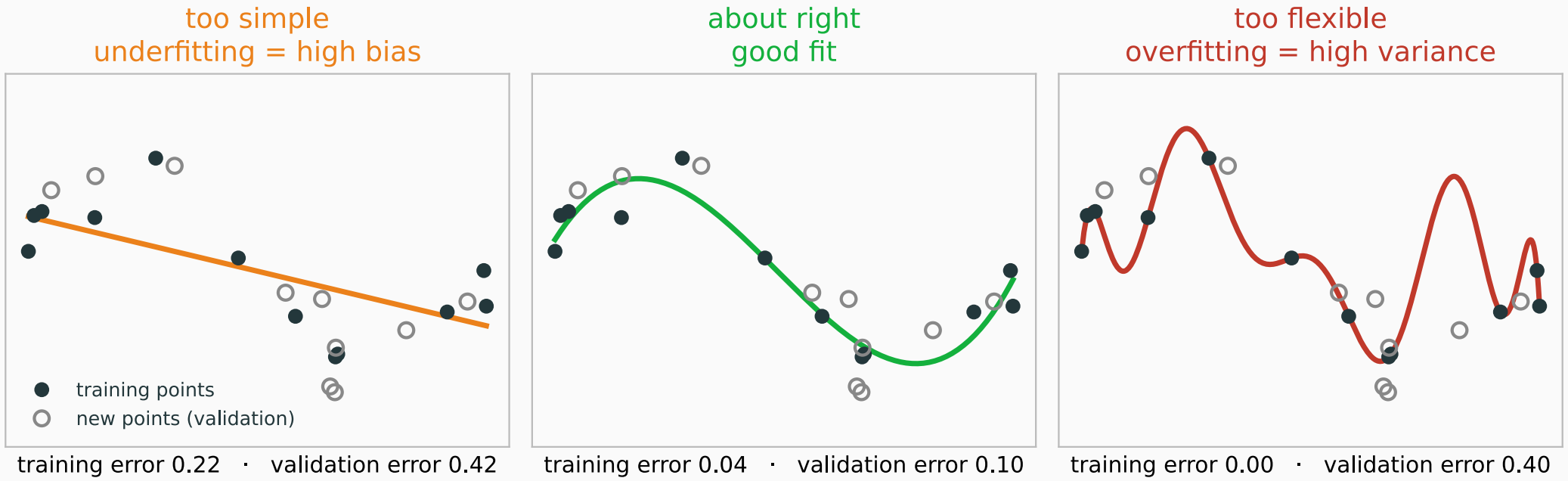
🔑 Key takeaway

The task decides the loss. Start with AdamW and `lr=1e-3`.
If training misbehaves, check the learning rate first - and turn one knob at a time.

Generalisation: working on new data

Tools against overfitting - and where they sit in the code

Generalisation: underfitting and overfitting

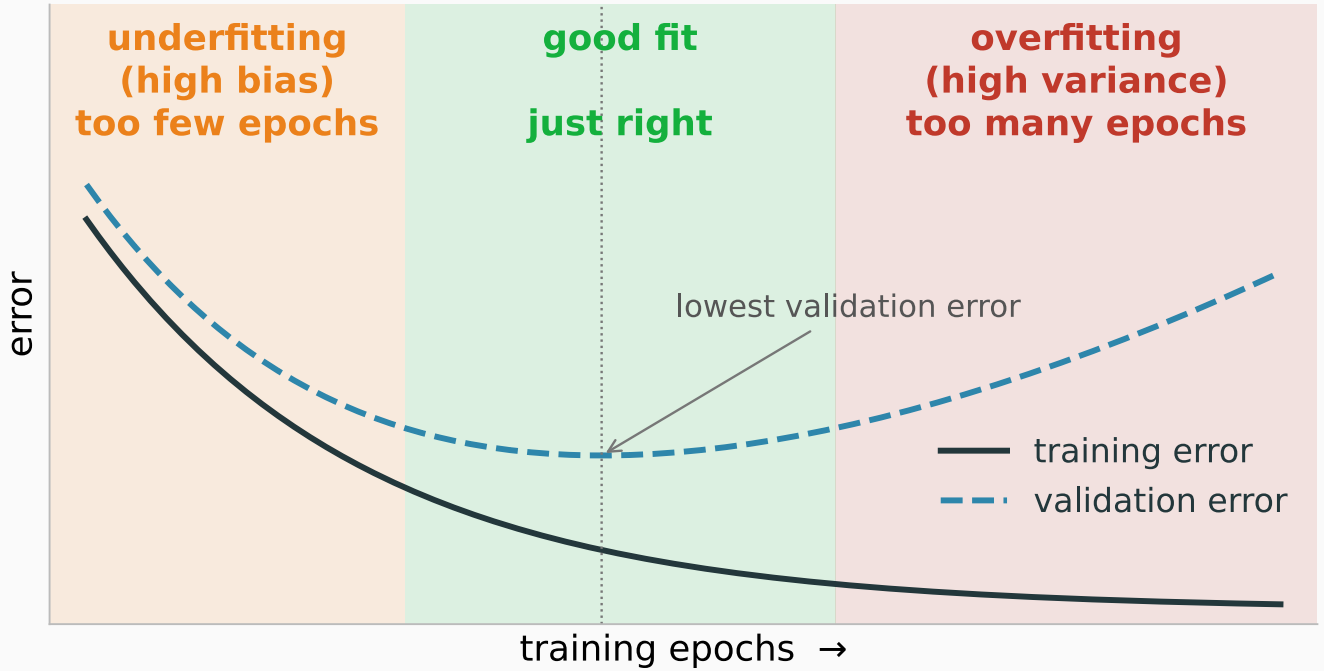


- **train high, val high** → underfitting (high bias): bigger model, train longer, check the learning rate
- **train low, val high** → overfitting (high variance): more data, augmentation, regularisation (next slides)

❓ Think about it

Training sensitivity 99 %, on a held-out record 60 %. Under- or overfitting?

Training and validation error: three regions



(schematic)

- the longer we train, the lower the **training error**
- the **validation error** goes down - then up again: the model starts to learn the noise
- stop at the epoch where the validation error is lowest → **early stopping** (later today)

The generalisation toolbox: where it sits in the code

```
train_ds = datasets.FashionMNIST(
    "data", train=True, download=True,
    transform=train_tf)    # augmentation
model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(784, 256),
    nn.BatchNorm1d(256),   # BatchNorm
    nn.ReLU(),
    nn.Dropout(p=0.3),     # dropout
    nn.Linear(256, 10))    # init: PyTorch default
opt = torch.optim.AdamW(
    model.parameters(), lr=1e-3,
    weight_decay=0.01)     # weight decay
```

```
best_val, wait = float("inf"), 0
for epoch in range(100):
    model.train()             # dropout ON
    for x, y in train_loader:
        loss = loss_fn(model(x), y)
        opt.zero_grad()
        loss.backward()
        opt.step()
    model.eval()              # dropout OFF
    with torch.no_grad():
        val_loss = loss_fn(model(X_val), y_val)
    if val_loss < best_val:  # early stopping
        best_val, wait = val_loss, 0
        torch.save(model.state_dict(), "best.pt")
    else:
        wait += 1
        if wait == 10:
            break
```

Weight decay: keep the weights small

- big weights → the model reacts strongly to tiny changes of the input → wiggly decision boundary → overfitting
- **weight decay**: in every step all weights shrink a little - only weights that the data really needs stay big
- example: `lr=0.1`, `weight_decay=0.01` → every step each weight is multiplied by 0.999 (on top of the normal update)
- it is one argument in the optimiser (knob ③):

```
opt = torch.optim.AdamW(model.parameters(), lr=1e-3,
                        weight_decay=0.01)
```

- typical values: 0.01 (AdamW), 0.0005 (SGD)
- also called L2 regularisation

🔑 Key takeaway

Weight decay is cheap and almost always helps a little: use AdamW with `weight_decay=0.01` as your default.

💡 Also exists: L1 regularisation

Punishes $|w|$ instead of w^2 → many weights become exactly **0**. Rare in deep learning - try it in TF Playground (next slide).

Try it: overfitting and regularisation in TensorFlow Playground

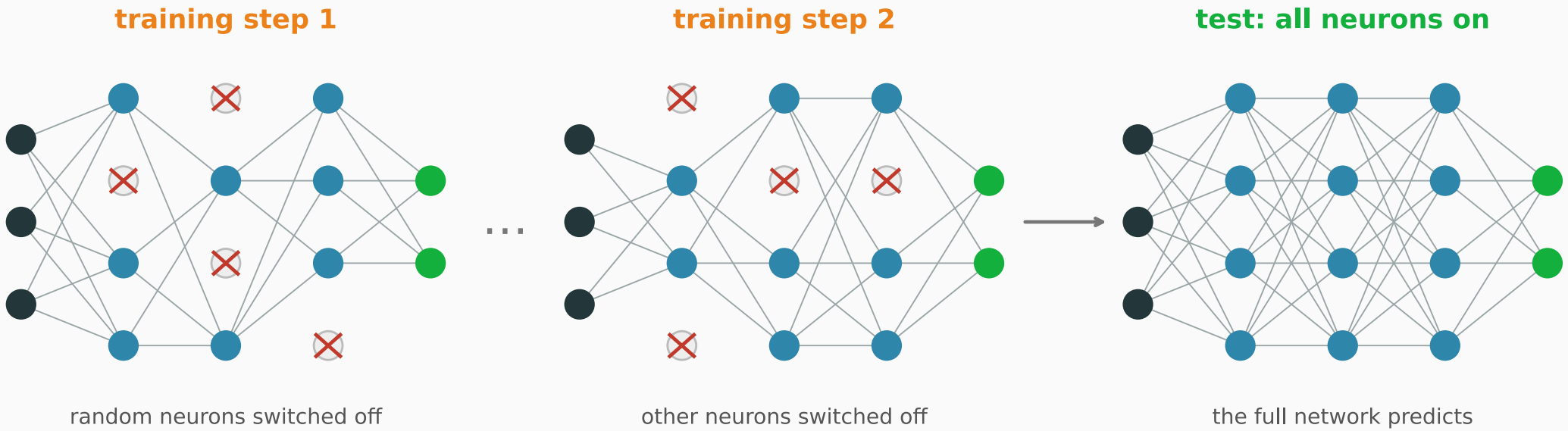
playground.tensorflow.org - dataset Spiral (or Circle)

1. **ratio of training to test data**: 10 %, **noise**: 50
2. a big network: 4 hidden layers with 8 neurons. Press play for about 500 epochs. Compare training loss and test loss. Look at the boundary.
3. reset, **regularization: L2**, **regularization rate: 0.01**. Play again. What changes?
4. instead of L2: ratio of training data back to 50 %. What happens now?

👥 Your turn (10 min)

In pairs, in your browser. Write down training loss and test loss after each step.

Dropout: switch off random neurons while training



- **training**: in every step a random part of the neurons is switched off (e.g. 30 %) - every step a different part
- the network cannot rely on single neurons - like a team where anybody could be missing that day
- **test**: all neurons are on

```
nn.Dropout(p=0.3)   # switch off 30 % of the neurons
```

- typical `p` = 0.1-0.5, mostly after fully connected layers
- PyTorch takes care of the scaling between training and test

Srivastava et al., JMLR 2014

BatchNorm: bring every feature to the same scale

One feature (e.g. alpha band power) in a batch of 4 EEG windows:

	window 1	window 2	window 3	window 4
value	120	80	100	100
minus mean (100)	20	−20	0	0
divided by spread (14)	**1.4**	**−1.4**	**0**	**0**

- like feature scaling from Day 1 - but inside the network, after a layer
- then the layer learns its own best scale and shift

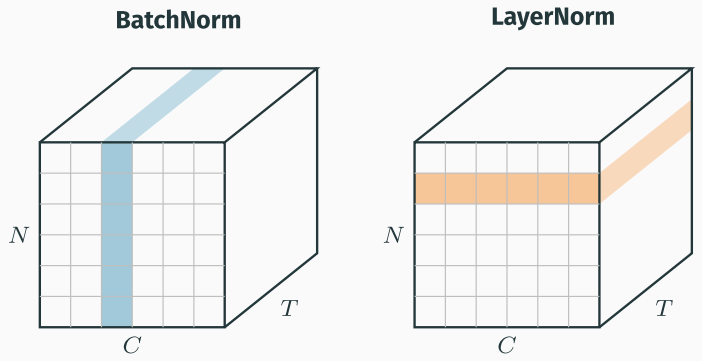
- **why?** every layer gets inputs of a similar size → training is more stable, larger learning rates work, less sensitive to initialisation
- one layer in the model:

```
nn.Linear(8, 64),
nn.BatchNorm1d(64),   # 64 features
nn.ReLU(),
```

- during training: mean and spread of the current batch; at test time: stored averages from training → `model.eval()` matters

Ioffe & Szegedy, ICML 2015

BatchNorm vs LayerNorm: average over what?



data cube: batch (N) × channels (C) × time (T); coloured = averaged together

	BatchNorm	LayerNorm
averages	one feature over all windows of the batch	all features of one window
needs a batch?	yes (≥ 16)	no
train = test?	no → `model.eval()`	yes
used in	CNNs (Day 4)	transformers, RNNs

🧠 EEG corner

BatchNorm mixes the windows of different recordings and patients; a new patient looks different → domain shift (Day 5).

figure style after Wu & He, ECCV 2018

Train mode vs eval mode

```
model.train()          # dropout ON, BatchNorm: current batch
for x, y in train_loader:
    loss = loss_fn(model(x), y)
    opt.zero_grad()
    loss.backward()
    opt.step()

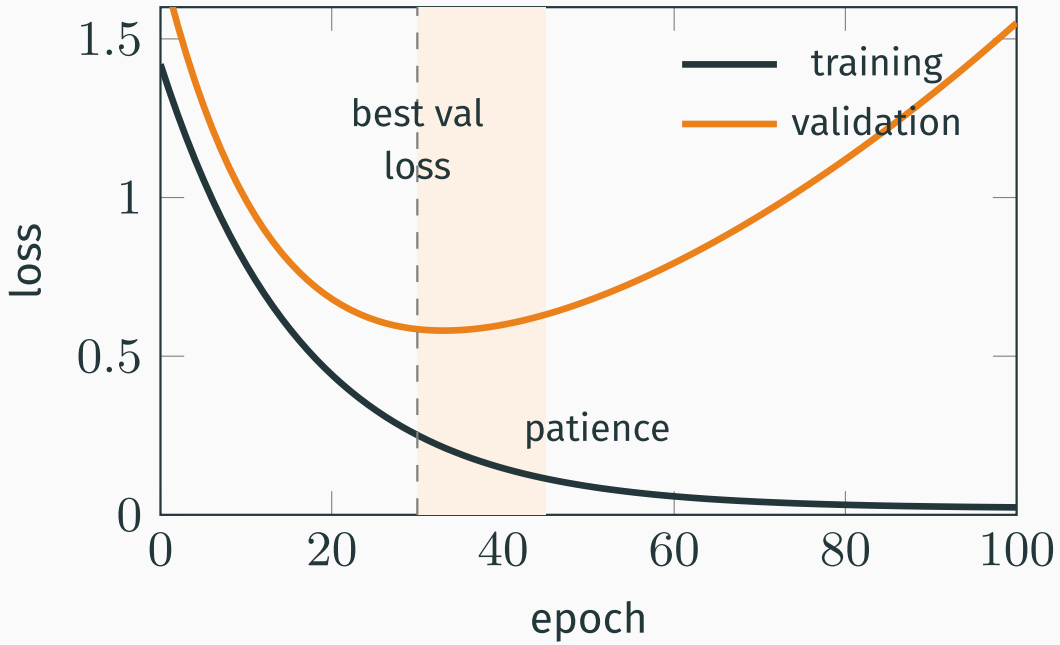
model.eval()           # dropout OFF, BatchNorm: stored averages
with torch.no_grad():  # no gradients - does NOT switch the mode!
    logits = model(x_val)
```

- `train()` / `eval()` change how dropout and BatchNorm behave
- `no_grad()` only switches off the gradient bookkeeping - for evaluation you need both

❓ Think about it

You forget `model.eval()` before validation. What do you see (a) with dropout, (b) with BatchNorm?

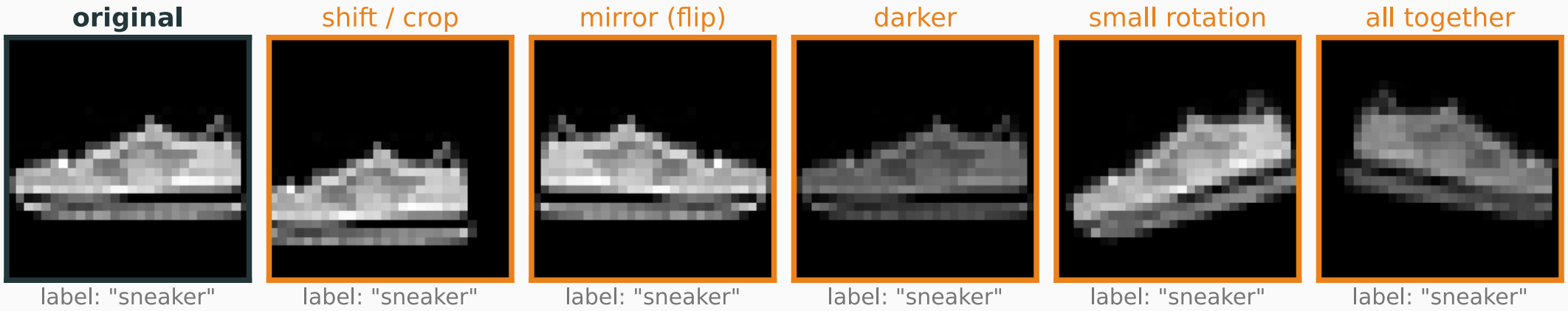
Early stopping



- after every epoch: compute the validation loss
- remember the weights of the best epoch so far
- no improvement for patience epochs (e.g. 5)?
  → stop and go back to the best weights
- the best epoch was chosen on the validation set → report the result on the test set

Prechelt, in: Neural Networks: Tricks of the Trade, 1998

Data augmentation: images




```
from torchvision.transforms import v2
train_tf = v2.Compose([v2.ToImage(),
    v2.RandomCrop(28, padding=3),    # shift
    v2.RandomHorizontalFlip(),       # mirror
    v2.RandomRotation(15),           # rotate
    v2.ColorJitter(brightness=0.4),  # darker
    v2.ToDtype(torch.float32, scale=True)])
```

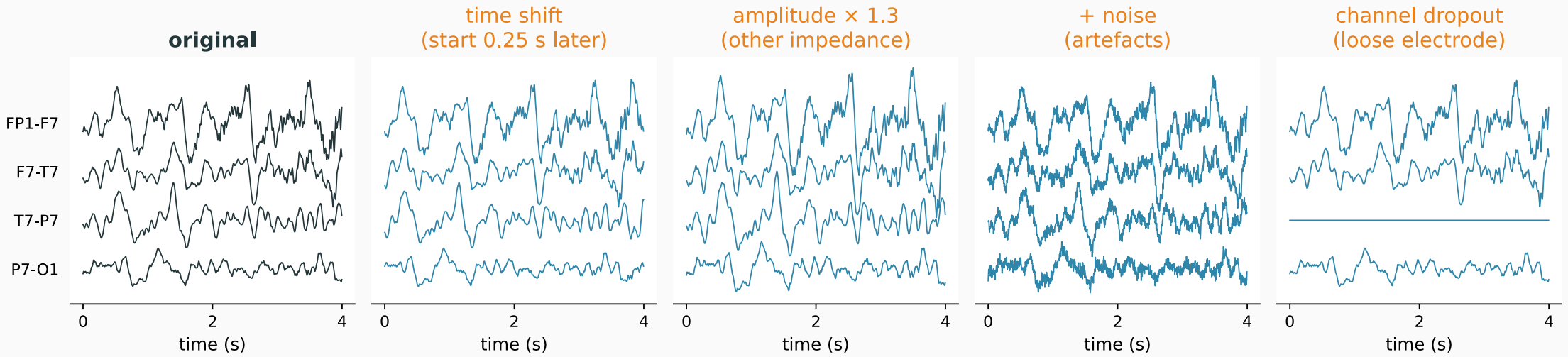
- one image → many new training images, the label stays the same
- a new random version in every epoch
- only the training data - validation and test stay untouched

❓ Think about it

Is a mirror (flip) OK for (a) FashionMNIST, (b) handwritten digits, (c) chest X-rays, (d) scanned documents?

FashionMNIST, training image of class "sneaker"

Data augmentation: EEG



every window keeps its label: "seizure"

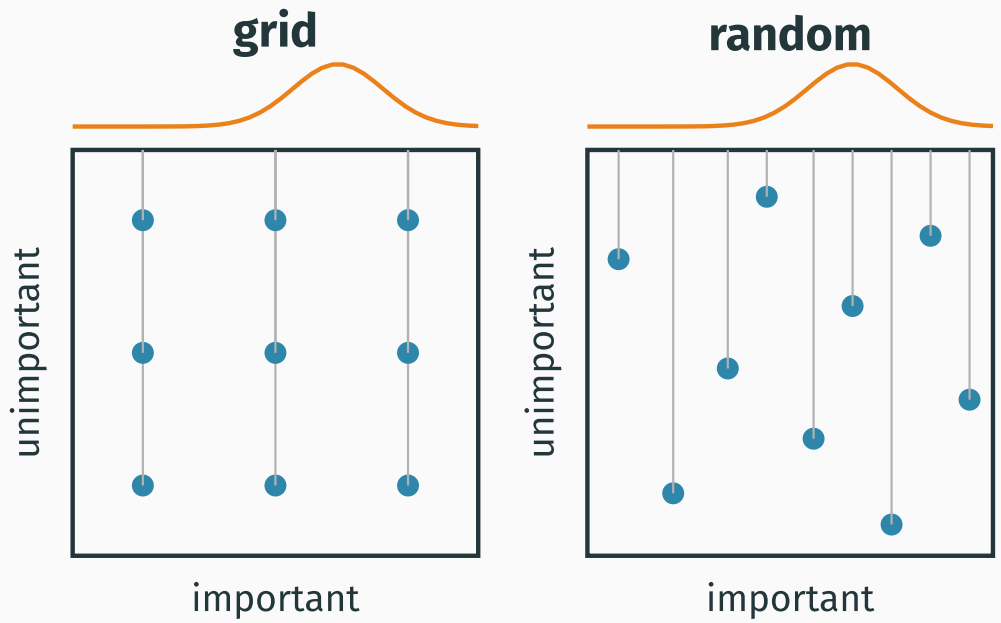
```python
def augment(x):                # x: one window (channels, time)
    shift = int(torch.randint(-64, 65, (1,)))
    x = torch.roll(x, shift, dims=-1)        # time shift
    x = x * torch.empty(1).uniform_(0.8, 1.2)     # amplitude
    x = x + 0.05 * x.std() * torch.randn_like(x)  # noise
    return x
```

🧠 EEG corner

The label must survive: a large time shift can move the seizure out of the window; a left-right flip changes the side of a focal seizure.

chb01_03, seizure at 3005 s, 4 of 23 channels; Rommel et al., J. Neural Eng. 2022

Hyper-parameter tuning: grid vs random search



9 trials each: the grid tests only 3 values of the important knob, random search 9

- hyper-parameters = the knobs you choose (learning rate, weight decay, dropout, batch size, ...)
- few matter a lot (learning rate first!), most matter little
- **random search** tries a new value of every knob in every trial → finds the good region faster than a grid
- try values on a log scale: 0.1, 0.01, 0.001 - not 0.1, 0.2, 0.3
- start wide with short runs, then narrow with longer runs
- tools that run the search for you: **Optuna**, Ray Tune, W&B Sweeps

Bergstra & Bengio, JMLR 2012

Keep track of your runs: TensorBoard, W&B, MLflow

After 30 runs nobody remembers which settings gave which result → **log every run automatically**: settings, seed, loss curves, metrics, git commit.

tool	what you get	in Colab
TensorBoard	live loss curves	built in: `%load_ext tensorboard`
Weights & Biases (W&B)	online dashboard, comparison tables, sweeps	free account
MLflow	open source, runs and models stored locally	UI best on your own laptop

```
from torch.utils.tensorboard import SummaryWriter
writer = SummaryWriter("runs/lr1e-3_seed0")
for epoch in range(epochs):
    ...                          # train + validate
    writer.add_scalar("loss/val", val_loss, epoch)
writer.close()
# Colab: %load_ext tensorboard
#        %tensorboard --logdir runs
```

🔑 Key takeaway

Project tip: choose one tool when you start coding - it also gives you the results table for your paper.

Cheat sheet: the generalisation toolbox

tool	where in the code	good first choice	use it when ...
weight decay	`AdamW(..., weight_decay=)`	0.01	always - it is cheap
dropout	`nn.Dropout(p)` in the model	0.1 - 0.3	training loss ≪ validation loss
BatchNorm / LayerNorm	`nn.BatchNorm1d` / `nn.LayerNorm` in the model	BatchNorm for CNNs	deep networks train slowly or unstably
train / eval mode	`model.train()` / `model.eval()`	always switch	before every validation and test
early stopping	`if` in the epoch loop	patience 5 - 10	always - keep the best weights
data augmentation	`transform=` in the dataset	shift, flip, noise	little data, overfitting
tuning + tracking	Optuna, W&B, MLflow	random search, log scale	after the baseline works

🔑 Key takeaway

Overfitting? More data or augmentation first, then weight decay, dropout and early stopping.
Make every choice on validation data - never on the test set.

Evaluation: does it really work?

Honest splits, the right metrics - and how sure we are

Which question does your test answer? The split decides

how you split	the test data is …	what you can claim
random windows	other windows of the same recordings	nothing - leakage (next slides)
by recording	new recordings of known patients	works for this patient (our labs: chb01)
by patient	new patients	works for a new patient
by hospital	another hospital, device, protocol	works elsewhere (Day 5)
by time	data recorded later	still works next year

🔑 Key takeaway

First write down the real-world question ("a new patient walks in"). Then split so that the test data looks like exactly that situation.

Shoeb & Guttag, ICML 2010 (patient-specific CHB-MIT detection)

Train, validation, test - and cross-validation

hold-out split



5-fold CV (folds = records / patients)



- **train** = homework: the model learns its weights
- **validation** = mock exam: we choose epochs, knobs, threshold
- **test** = final exam: the final number, used once at the end
- **cross-validation**: rotate the test part - every recording is test once → report mean ± std
- `GroupKFold` in scikit-learn: group = recording or patient, so one recording never ends up in training and test

Data leakage: seeing the exam questions in advance

Leakage = information from the test data sneaks into the training or into your choices → the numbers look great, but the model fails in real life.

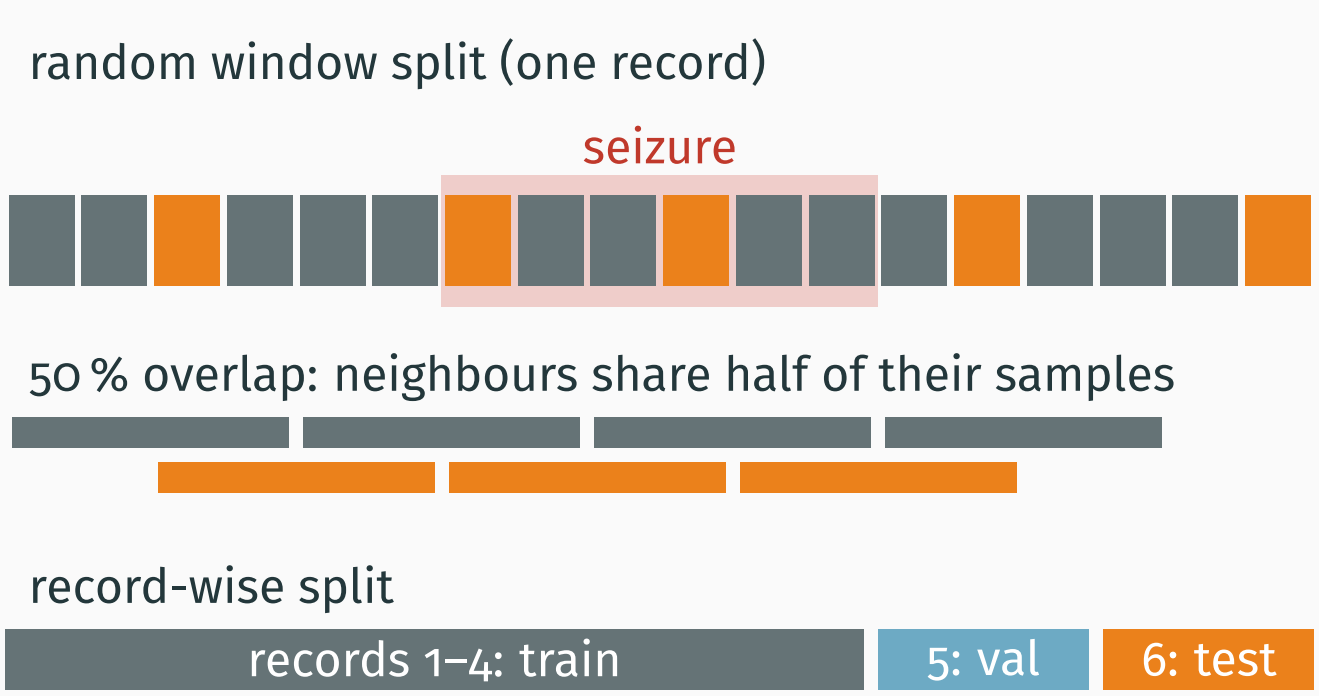
1. **the same recording or patient in training and test** - e.g. a random window split
2. **preprocessing computed on all data** - normalisation, feature selection, oversampling before the split
3. **choices made by looking at test results** - epochs, threshold, seed
4. **windows that overlap the border** between training and test

❓ Think about it

A paper: 99.8 % sensitivity on CHB-MIT, "10-fold CV on 2-s windows, 50 % overlap". What is your first question?

Kapoor & Narayanan, Patterns 2023

Why random window splits leak in EEG



- neighbouring EEG windows look almost the same - a seizure changes slowly
- random split: for almost every test window there is a near-copy in training → the model recognises this seizure, not seizures in general
- 50 % overlap: neighbours even share half of their samples

🧠 EEG corner

Lab: same model, same data - random vs record-wise split. You will measure the gap.

Preprocessing leakage

```
# WRONG: mean and std of ALL windows (incl. test)
X = (X - X.mean(0)) / X.std(0)
X_tr, X_te = X[rec != 5], X[rec == 5]

# RIGHT: compute them on the training part only
X_tr, X_te = X[rec != 5], X[rec == 5]
mu, sd = X_tr.mean(0), X_tr.std(0)
X_tr, X_te = (X_tr - mu) / sd, (X_te - mu) / sd
```

- the rule holds for every step that learns from data: normalisation, feature selection, oversampling, PCA
- fixed steps (band-pass filter, resampling) are safe
- lock the test set away until the very end

The confusion matrix and its metrics

	predicted seizure	predicted background
true seizure	**TP** seizure found	**FN** seizure missed
true background	**FP** false alarm	**TN** left alone

metric	question	=
sensitivity (recall)	how many seizures did we find?	TP / (TP + FN)
specificity	how much background did we leave alone?	TN / (TN + FP)
precision	how many alarms were real?	TP / (TP + FP)
accuracy	how many windows are right?	(TP + TN) / all
F1	high only if precision and sensitivity are high	
balanced acc.	average of sensitivity and specificity	

`sklearn.metrics.confusion_matrix(y_true, y_pred)` • `classification_report(y_true, y_pred)`

👥 Your turn: compute the metrics

100 EEG windows: **10 seizure**, **90 background**. Three models:

model A

	pred. seizure	pred. background
true seizure	TP **8**	FN **2**
true background	FP **9**	TN **81**

model B: "always background"

	pred. seizure	pred. background
true seizure	TP **0**	FN **10**
true background	FP **0**	TN **90**

model C: "always seizure"

	pred. seizure	pred. background
true seizure	TP **10**	FN **0**
true background	FP **90**	TN **0**

Compute for each model: **accuracy, sensitivity, specificity, precision**.
Which model is best? Which metric would choose the wrong model?

👥 In pairs (10 min)

The formulas are on the previous slide. A calculator is fine - the numbers are easy.

Solution: every single metric can be fooled

	model A	B: "always background"	C: "always seizure"
accuracy	89 %	**90 %** 😱	10 %
sensitivity	80 %	0 %	**100 %** 😱
specificity	90 %	**100 %** 😱	0 %
precision	47 %	- (0 / 0)	10 %
F1	0.59	0	0.18
balanced accuracy	**85 %**	50 %	50 %

- B is useless - but has the highest accuracy and a perfect specificity
- C is useless - but has a perfect sensitivity
- only metrics that look at both classes (F1, balanced accuracy, sensitivity with specificity) see that A is best

🔑 Key takeaway

Never report one metric alone. Report sensitivity and specificity (or precision) - and for seizures: false alarms per hour.

Real numbers: what does 99 % specificity mean?

❓ Think about it

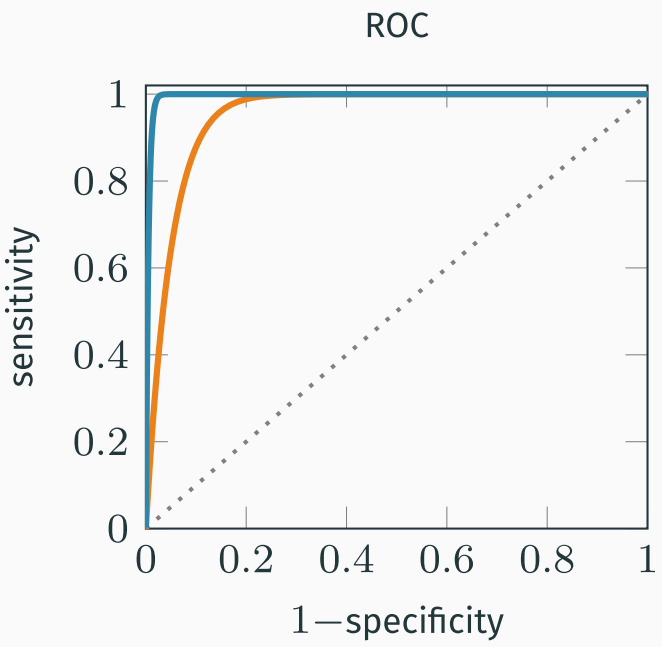
A window classifier on chb01 (86 seizure, 4995 background windows of 4 s) has 90 % sensitivity and 99 % specificity.
What are its accuracy and precision? How many false alarms per hour (if every false positive is a separate alarm)?

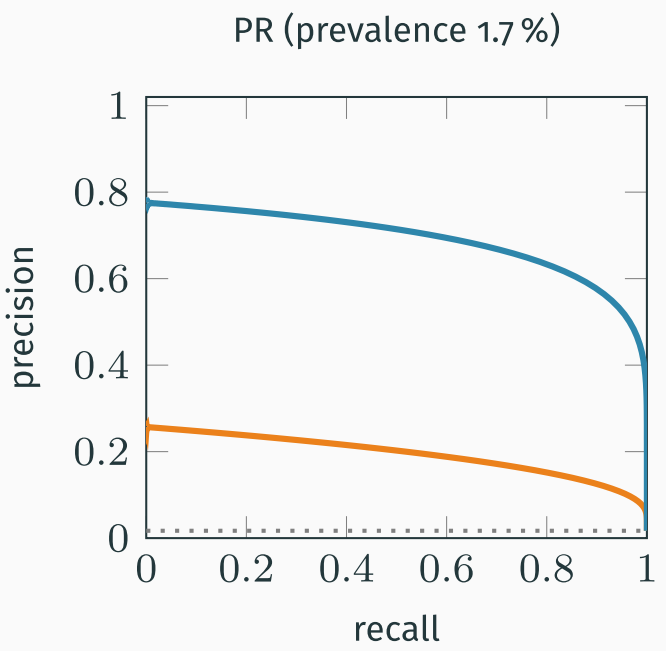
- TP $= 0.9 \cdot 86 \approx 77$, FP $= 0.01 \cdot 4995 \approx 50$
- accuracy $= (77 + 4945)/5081 \approx 98.8\,\%$ - "always background" scores 98.3 %
- precision $= 77/(77 + 50) \approx 0.61$ - 4 of 10 alarms are false
- background time $= 4995 \cdot 4\,\mathrm{s} \approx 5.5\,\mathrm{h} \Rightarrow \approx 9$ false alarms per hour

🔑 Key takeaway

Under heavy imbalance accuracy hides everything: report sensitivity with specificity, precision and false alarms.

Without a threshold: ROC and precision-recall curves





AUROC 0.95
AUROC 0.995 (schematic)

- the model gives a score; every threshold gives another confusion matrix = one point on the curve
- **AUROC** (area under the ROC curve): 0.5 = guessing, 1 = perfect
- **AUPRC** (area under the PR curve): guessing = share of seizures (1.7 % in chb01) → much more honest under imbalance
- report both

Saito & Rehmsmeier, PLoS ONE 2015

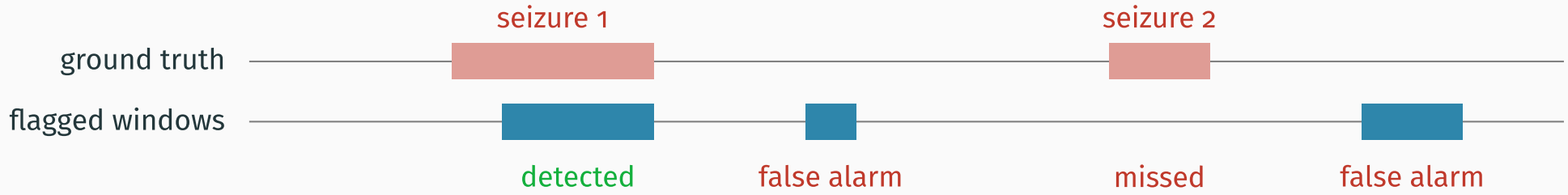
Choosing the decision threshold

- the threshold turns the score into yes / no - it belongs to the model
- low threshold: more seizures found - but more false alarms; high threshold: fewer alarms - but more missed seizures
- the right balance is a clinical decision
- choose the threshold on the validation data, then test once with that fixed threshold
- after a weighted loss (block 1) 0.5 is not a special threshold any more

❓ Think about it

The epileptologist says: "at most one false alarm per day, otherwise nobody wears the device".
How do you set the threshold - and with which data?

Seizure metrics: events and false alarms per hour



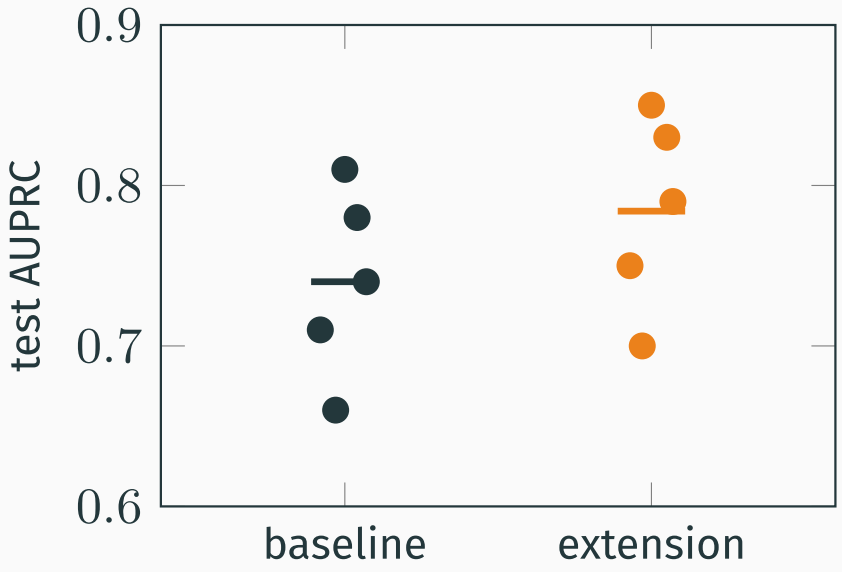
- **event sensitivity** = detected seizures / all seizures; a seizure counts as detected if at least one window inside is flagged (here 1 of 2)
- **false alarms per hour** = false-alarm events / hours of background; flagged windows in a row = one alarm
- **latency**: time from seizure start to the first alarm
- smoothing ("alarm only if 3 of 5 windows are flagged") → fewer false alarms, but later and less sensitive

🧠 EEG corner

Window metrics answer "how many 4-s windows are right?" - clinicians ask "did it catch the seizure, and how often does it ring for nothing?".
SzCORE proposes a standard for this scoring.

Dan et al., Epilepsia 2024 (SzCORE)

One run is one sample: variance across seeds



(illustrative numbers)

- randomness everywhere: initialisation, batch order, dropout, augmentation
- the spread between seeds of the same model can be as large as a "new method's" improvement
- report mean ± std over at least 5 seeds and show the single runs
- fix the seeds for reproducibility - never pick the best seed

Bouthillier et al., MLSys 2021

How sure are we? Bootstrap confidence intervals

1. take your test set (n windows - better: n recordings or patients)
2. draw n of them at random with replacement (some twice, some not at all) and compute the metric
3. repeat 1000-2000 times → 1000 slightly different results
4. the middle 95 % of them = the **95 % confidence interval**

- works for every metric (AUPRC, F1, false alarms per hour)
- comparing two models: compute the difference on the same draws - if its 95 % interval does not include 0, the difference is unlikely to be luck

```
torch.manual_seed(0)
stats = []
for _ in range(2000):
    idx = torch.randint(0, len(y), (len(y),))
    if y[idx].sum() == 0:   # no seizure drawn
        continue
    stats.append(average_precision_score(
        y[idx], p[idx]))  # scikit-learn: metrics only
lo, hi = torch.quantile(torch.tensor(stats),
                        torch.tensor([0.025, 0.975]))
```

Efron & Tibshirani, An Introduction to the Bootstrap, 1993

Evaluation checklist for your project

Question	e.g. "detects seizures in new patients"
Split	by patient (or recording); train / validation / test; test used once
Preprocessing	everything that learns from data is fitted on the training data only
Choices	knobs, epochs, threshold on validation data
Metrics	one main metric chosen in advance (e.g. AUPRC or event sensitivity at ≤ x false alarms/h) + sensitivity, specificity, F1, false alarms/h
Repetitions	at least 5 seeds; mean ± std, show all runs
Uncertainty	95 % bootstrap confidence interval (of the difference, when you compare models)
Reporting	table + per-patient results; code and seeds in Git

Red flags when you read a paper

- windows split randomly; "k-fold CV" without saying over what
- normalisation or feature selection before the split
- accuracy as the only metric on imbalanced data
- threshold, epoch or model chosen on the test set
- one run only - no std, no confidence interval
- baseline numbers copied from another paper with a different split

🔑 Key takeaway

Define the question, split by the unit you want to generalise to, fit everything on training data, choose on validation data, report several metrics with their uncertainty.

Lab 3: Training & Evaluation in Practice

Lab 3: write it yourself

💻 `day3_training_evaluation.ipynb` in Colab · CPU is enough · first run: about 2 min download

Part 1 - build it yourself (FashionMNIST, ~30 min)

- 1.1 a model from a plan: `nn.Sequential` with 6 layers
- 1.2 your own training pipeline: training loop + validation

Part 2 - grid search (~20 min)

- 2.1 turn the model into `make_model(hidden)`
- 2.2 3 learning rates × 3 sizes: choose on validation, test once

Part 3 - the right metric in the right place (CHB-MIT chb01, ~25 min)

- 3.1 an EEG model from a plan
- 3.2 compute the metrics yourself with scikit-learn
- 3.3 choose the best model by accuracy - and by AUPRC
- 3.4 which metric where?

👥 How it works

In pairs. Every task: plan → write the code (numbered steps) → 💡 hint if stuck → ✅/❌ check. What you do not finish is homework.

Wrap-up

Summary

- **Training loop**: the task decides the loss; imbalance → class weights; AdamW + `lr=1e-3` as a start; the learning rate is the most important knob (try values a factor 3-10 apart; cosine schedule)
- **Generalisation**: training vs validation loss tells under- from overfitting; weight decay, dropout (train vs eval mode), early stopping, augmentation; BatchNorm
- **Tuning**: random search on log scales, on validation data; track every run (TensorBoard, W&B, MLflow)
- **Evaluation**: split by the unit of generalisation; no leakage; never one metric alone - sensitivity and specificity, AUPRC, event sensitivity, false alarms/h
- **Statistics**: at least 5 seeds, mean ± std, bootstrap confidence intervals

Lab 3 – Write it yourself

Notebook: `day3_training_evaluation.ipynb` (course repository) · open it in Colab · CPU is enough · work in **pairs** · what you do not finish in class is **homework**.

How every writing task works

1. read the **plan** (a small table on this sheet and in the notebook),
2. write your code below the numbered steps – where it says `pass  # YOUR CODE HERE`,
3. stuck? open **Hint** in the notebook,
4. run the **check cell**: ✔ = correct, ✘ = it tells you what is wrong,
5. tick the box ☐ of the task on this sheet.

First run the two **setup cells**. Something looks strange? *Runtime* → *Restart session*, then the setup cells again.

Part 1 – Build it yourself

FashionMNIST · about 30 min

☐ 1.1 A model from a plan

10 min

Build this network with `nn.Sequential(...)` – the layers in exactly this order:

#	layer	in	out	note
1	`nn.Linear`	784	256	one pixel value per input
2	`nn.ReLU`			
3	`nn.Dropout`			switch off 20 % of the neurons (`p=0.2`)
4	`nn.Linear`	256	128	
5	`nn.ReLU`			
6	`nn.Linear`	128	10	one raw score (logit) per kind of clothes – no softmax!

Check: layers and sizes, **235 146** parameters.

Question: where do the 235 146 parameters come from? Compute the first layer by hand (weights + biases).

☐ 1.2 Your own training pipeline

20 min

Write the body of `train_model(model, lr, epochs)`. It trains the model and returns the **validation accuracy**. You know every line from the slide *"The training loop: seven knobs"*.

step	what	PyTorch
1	optimiser	`torch.optim.Adam(model.parameters(), lr=lr)`
2	repeat for every epoch	`for epoch in range(epochs):` + `model.train()`
3	repeat for every batch	`for x, y in loader:`
4	the 5 training lines	forward → loss → `zero_grad()` → `backward()` → `step()`
5	validation	`model.eval()`, `with torch.no_grad():`, predictions for `X_val`
6	accuracy	share of correct predictions (a number between 0 and 1)

Check: validation accuracy $\geq$ 0.80 after 3 epochs.

Try: `LR = 0.1` – what happens? Then `EPOCHS = 10` (with `LR = 0.001`).

Part 2 – Grid search

FashionMNIST · about 20 min

☐ 2.1 A model "factory"

5 min

Write `make_model(hidden)`: the model from 1.1, but the first hidden layer has `hidden` neurons instead of 256 (layer 1: 784 → `hidden`, layer 4: `hidden` → 128).
Check: works for `hidden = 32` and `hidden = 512`.

☐ 2.2 The grid search

15 min

1. two loops, one inside the other: for every learning rate in `[0.0001, 0.001, 0.01]`, for every size in `[32, 128, 512]` → build the model, train it, add `{"lr": ..., "hidden": ..., "val_acc": ...}` to `results`
2. pick the entry with the **highest validation accuracy**
3. only now – **once** – measure the best setting on the **test** images

Check: 9 results, the best one chosen by validation accuracy.
Question: why do we choose the best setting with the *validation* images and not with the test images?

Part 3 – The right metric in the right place

CHB-MIT chb01 · about 25 min

EEG of patient chb01: 4-s windows, 90 band-power numbers per window, split **by recording** (4 training, 1 validation, 1 test). The download takes about 1–2 minutes.

☐ 3.1 An EEG model from a plan

3 min

Write `make_eeg_model()`: `nn.Linear(90, 32)` → `nn.ReLU` → `nn.Dropout(p=0.3)` → `nn.Linear(32, 2)` (two classes: background, seizure). Training is done by the given helper `train_eeg(...)`.

☐ 3.2 Compute the metrics yourself

10 min

Compute five metrics for the validation recording with scikit-learn (true labels: `y_val_eeg`):

metric	scikit-learn	uses
accuracy	`accuracy_score(y_true, y_pred)`	`pred`
sensitivity	`recall_score(y_true, y_pred)`	`pred`
specificity	`recall_score(y_true, y_pred, pos_label=0)`	`pred`
precision	`precision_score(y_true, y_pred)`	`pred`
AUPRC	`average_precision_score(y_true, scores)`	`scores` (no threshold!)

Try: `WEIGHT_SEIZURE = 58` – one seizure window counts like 58 background windows.
Question: which numbers tell you something useful here – and which one hardly moves?

☐ 3.3 Which metric chooses the model? 7 min

A helper trains 6 models (3 class weights × 2 learning rates). Choose the best one **twice**: once by accuracy, once by AUPRC (the same loop as in 2.2).
Question: which of the two models would you give to a doctor – and why?

☐ 3.4 Which metric where? 5 min

Fill in the right column (in the notebook or here). Use the slides of block 3.

situation	which metric(s)?
choose the best FashionMNIST model (10 equally common classes)	
choose the best seizure model in a grid search	
choose the threshold of the seizure detector	
tell a doctor how good the detector is	
the results table of your project	

☐ ★ Bonus How sure are we about the test AUPRC? homework

Bootstrap from the slides: draw the test windows 2000 times with replacement and compute the AUPRC each time. Fill in one line: add the AUPRC of each draw to the list `draws`.